

ASSIGNMENT-12.3

NAME: G. MANI SHARAN REDDY

BATCH: 41

HALL.NO: 2303A52148

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates

efficiently during campus placements. Student records must be sorted by

CGPA in descending order.

Prompt:

Generate a Python program to store student records (Name, Roll Number, CGPA). Implement Quick Sort and Merge Sort to sort students by CGPA in descending order. Measure runtime for large datasets and display top 10 students

Code:

```

1  import random
2  import string
3  import time
4  from copy import deepcopy
5
6  # -----
7  # Data Generation
8  # -----
9
10 def generate_students(n):
11     students = []
12     for i in range(n):
13         name = ''.join(random.choices(string.ascii_uppercase, k=5))
14         roll = f"SRU{i:05d}"
15         cgpa = round(random.uniform(5.0, 10.0), 2)
16         students.append({"name": name, "roll": roll, "cgpa": cgpa})
17     return students
18
19 # -----
20 # Quick Sort
21 # -----
22
23 def quick_sort(arr):
24     if len(arr) <= 1:
25         return arr
26     pivot = arr[len(arr)//2]["cgpa"]
27     left = [x for x in arr if x["cgpa"] > pivot]
28     middle = [x for x in arr if x["cgpa"] == pivot]
29     right = [x for x in arr if x["cgpa"] < pivot]
30     return quick_sort(left) + middle + quick_sort(right)
31
32

```

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i]["cgpa"] > right[j]["cgpa"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# -----
# Top 10 Display
# -----
```

```

def display_top_10(students):
    print("\nTop 10 Students:")
    for i, student in enumerate(students[:10]):
        print(f"{i+1}. {student['name']} | {student['roll']} | CGPA: {student['cgpa']}")

# -----
# Performance Comparison
# -----

students = generate_students(10000)

quick_data = deepcopy(students)
merge_data = deepcopy(students)

start = time.time()
quick_sorted = quick_sort(quick_data)
quick_time = time.time() - start

start = time.time()
merge_sorted = merge_sort(merge_data)
merge_time = time.time() - start

print(f"\nQuick Sort Time: {quick_time:.5f} seconds")
print(f"Merge Sort Time: {merge_time:.5f} seconds")

display_top_10(quick_sorted)

```

Output:

```
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> python 12.1.py
Quick Sort Time: 0.01451 seconds
Merge Sort Time: 0.02586 seconds

Top 10 Students:
1. Raj Patel | SRU00212 | CGPA: 10.0
2. Amit Kapoor | SRU00535 | CGPA: 10.0
3. Harsh Patel | SRU01273 | CGPA: 10.0
4. Amit Iyer | SRU04309 | CGPA: 10.0
5. Shruti Mishra | SRU05511 | CGPA: 10.0
6. Karan Nair | SRU05784 | CGPA: 10.0
7. Siddharth Iyer | SRU08788 | CGPA: 10.0
8. Akash Kumar | SRU09074 | CGPA: 10.0
9. Ishita Kapoor | SRU09585 | CGPA: 10.0
10. Varun Iyer | SRU09688 | CGPA: 10.0
PS C:\Users\2303A\OneDrive\3-2 subjects\ai>
```

Justification:

Performance Analysis

Algorithm	Best Case	Average Case	Worst Case
-----------	-----------	--------------	------------

Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
------------	---------------	---------------	----------

Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
------------	---------------	---------------	---------------

Justification:

Quick Sort performs faster in practice due to in-place behavior.

Merge Sort guarantees stable performance even for worst case

Task 2: Implementing Bubble Sort with AI Comments

Task: Write a Python implementation of Bubble Sort

Prompt:

Generate Bubble Sort in Python with inline explanatory comments and time complexity analysis

Code:

```
def bubble_sort(arr):
    n = len(arr)

    # Outer loop controls number of passes
    for i in range(n):

        swapped = False # Optimization to detect if array is already sorted

        # Inner loop compares adjacent elements
        for j in range(0, n - i - 1):

            # Swap if left element is greater than right element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swapping happened, array is sorted
        if not swapped:
            break

    return arr

# Example
data = [64, 34, 25, 12, 22, 11, 90]
print("Sorted:", bubble_sort(data))
```

Output:

```
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> & C:/Users/2303A/AppData/Local/
ers/2303A/OneDrive/3-2 subjects/ai/12.2.py"
Sorted: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\2303A\OneDrive\3-2 subjects\ai>
```

Justification:

Time Complexity Analysis

Best Case: $O(n)$ (already sorted)

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

Space Complexity: $O(1)$ (In-place)

Task 3: Quick Sort and Merge Sort Comparison

Task: Implement Quick Sort and Merge Sort using recursion

code:

```
def quicksort_recursive(arr):  
    """  
    Sorts a list using recursive Quick Sort.  
    Average Time Complexity:  $O(n \log n)$   
    Worst Case:  $O(n^2)$   
    """  
  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[-1]  
    left = [x for x in arr[:-1] if x <= pivot]  
    right = [x for x in arr[:-1] if x > pivot]  
  
    return quicksort_recursive(left) + [pivot] + quicksort_recursive(right)
```

Output:

```
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> & C:/Users/2303A/OneDrive/3-2 subjects/ai/12.3.py"  
[1, 1, 2, 3, 6, 8, 10]  
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> █
```

recursive merge sort:


```
def mergesort_recursive(arr):
    """
    Time complexity:  $O(n \log n)$  for all cases.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr)//2
    left = mergesort_recursive(arr[:mid])
    right = mergesort_recursive(arr[mid:])

    return merge_lists(left, right)

def merge_lists(left, right):
    result = []
    while left and right:
        if left[0] <= right[0]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))

    result.extend(left)
    result.extend(right)
    return result
```

Output:

```
[3, 9, 10, 27, 38, 43, 82]
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> █
```

Justification:

Complexity Summary

Input Type Quick Sort Merge Sort

Random $O(n \log n)$ $O(n \log n)$

Sorted $O(n^2)$ $O(n \log n)$

Reverse $O(n^2)$ $O(n \log n)$

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products,

each with attributes like product ID, name, price, and stock quantity. Store staff

need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Code:

```
inventory = [
    {"id": 101, "name": "Keyboard", "price": 750, "quantity": 30},
    {"id": 102, "name": "Mouse", "price": 500, "quantity": 50},
    {"id": 103, "name": "Monitor", "price": 8500, "quantity": 20},
]

# Hash Map for fast search
inventory_map = {item["id"]: item for item in inventory}

def search_by_id(product_id):
    return inventory_map.get(product_id, "Not Found")

def sort_by_price():
    return sorted(inventory, key=lambda x: x["price"])

print(search_by_id(102))
print(sort_by_price())
```

Output:

```
PS C:\Users\2303A\OneDrive\3-2 subjects\ai\12.3.py> python3 12.3.py
{"id": 102, "name": "Mouse", "price": 500, "quantity": 50}
[{"id": 102, "name": "Mouse", "price": 500, "quantity": 50}, {"id": 101, "name": "Keyboard", "price": 750, "quantity": 30}, {"id": 103, "name": "Monitor", "price": 8500, "quantity": 20}]
PS C:\Users\2303A\OneDrive\3-2 subjects\ai>
```

Justification:

Large dataset → Hash Map reduces search from $O(n)$ to $O(1)$

Retail requires frequent search → Constant-time lookup critical

Sorting less frequent → $O(n \log n)$ acceptable

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing

stock price movements. The requirement is to quickly sort stocks by daily

gain/loss and search for specific stock symbols efficiently.

Code: Heap Sort Implementation

```
import heapq

def rank_stocks(stocks):
    heap = []

    for stock in stocks:
        change = ((stock["close"] - stock["open"]) / stock["open"]) * 100
        heapq.heappush(heap, (-change, stock))

    ranked = []
    while heap:
        ranked.append(heapq.heappop(heap)[1])

    return ranked

stocks = [
    {"symbol": "TCS", "open": 3500, "close": 3600},
    {"symbol": "INFY", "open": 1400, "close": 1450},
]

stock_map = {s["symbol"]: s for s in stocks}

def search_stock(symbol):
    return stock_map.get(symbol.upper(), "Stock Not Found")

print(search_stock("TCS"))
```

Output:

```
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> & C:/Users/2303A/AppData/Local/Python/py
ers/2303A/OneDrive/3-2 subjects/ai/12.3.py
{'symbol': 'TCS', 'open': 3500, 'close': 3600}
PS C:\Users\2303A\OneDrive\3-2 subjects\ai> █
```

Justification:

Performance Comparison

Operation Heap Sort sorted()

Sorting $O(n \log n)$ $O(n \log n)$

Search $O(1)$ HashMap $O(1)$ Dict

Trade-offs

sorted() (Timsort) is highly optimized in CPython

Heap Sort useful when continuously extracting top-K elements

Hash Map ideal for instant symbol retrieval